USED MEDIAS LLC
EMBEDDED SYSTEMS DIVISION

System Architecture and Scope

UM-NATOS-001

DOCUMENT	REVISION	STATUS	CLASSIFICATION
UM-NATOS-001	1.0 · 2026-08-14	current	Internal

1. Abstract

nat-os is an operating system written from scratch for the ESP32-based "Cheap Yellow Display" (CYD) board. It does not use ESP-IDF, Arduino, or FreeRTOS. The kernel owns scheduling, memory, drivers, and application execution; only the second-stage bootloader and partition table are reused, and both are replaceable.

This report defines the scope of "from scratch" for this project, the layer model, and the two architectural decisions that constrain everything downstream: starting at layer L2, and running applications in a bytecode virtual machine.

2. Target hardware

Property	Value	Consequence for the design
SoC	ESP32-D0WD, dual-core Xtensa LX6 @ 240 MHz	Real SMP is possible; deferred until after M2
Internal SRAM	520 KB (SRAM0 192 KB / SRAM1 128 KB / SRAM2 200 KB)	Ample for a kernel; tight once a graphics stack is added
PSRAM	**None** (stock CYD board)	No external RAM to fall back on
Flash	4 MB, executed in place via cache	Code can exceed IRAM, but requires cache setup (deferred past M0)
MMU	Flash-cache address translation **only**	**No per-process virtual address spaces** — see §4
Display	ILI9341 240×320 with resistive touch	Not addressed before M5

3. Layer model and project scope

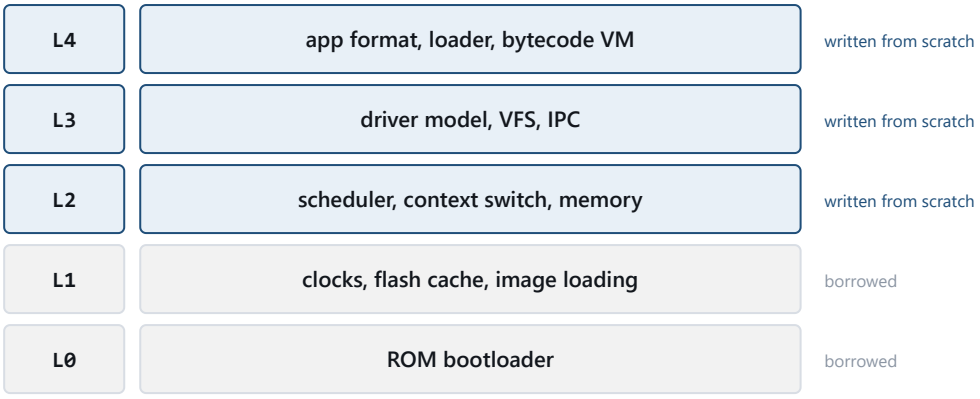

Figure 1. *Project scope by layer. Borrowing L1 costs one binary dependency and saves weeks of silicon bring-up.*

The system is divided into five layers. The scope decision is *where the project starts writing its own code*.

Layer	Contents	nat-os
L0	Mask-ROM bootloader	Silicon. Cannot be replaced.
L1	Second-stage bootloader: clock config, flash cache/MMU init, image loading	**Borrowed.** Replaceable later.
L2	Scheduler, context switching, synchronisation, memory allocation	**Written from scratch**
L3	Driver model, virtual filesystem, inter-process communication	**Written from scratch**
L4	Application format, loader, virtual machine, shell	**Written from scratch**

3.1 Rationale for an L2 start

Writing L1 means implementing clock trees, flash cache configuration, and SPI timing calibration against the technical reference manual. That is silicon bring-up, not operating system design. It is a legitimate project, but a different one, and it delays the first scheduler by weeks.

Borrowing L1 costs one binary dependency (17,536 bytes) and yields a running CPU with flash mapped and a well-defined handoff. Everything above that handoff is original work. The borrowed component is isolated behind a documented interface — the image header format (UM-NATOS-002) — so replacing it later is a contained change rather than a rewrite.

4. The isolation problem

The ESP32 cannot provide memory protection between applications. Its MMU translates flash addresses for execute-in-place caching; it does not implement per-process page tables. There is one address space, and any code can write any address.

This is a hardware property, not an implementation gap. It cannot be engineered around at the kernel level. It has a direct consequence: **a native-code application can corrupt the kernel and every other application, and no kernel design prevents this.**

Two responses were considered.

4.1 Option A — native applications via an ELF loader

Applications are compiled separately, stored on SD, loaded into heap at runtime, relocated, and executed natively.

- Fast — applications run at full CPU speed.
- Precedent exists on ESP32.
- **No isolation whatsoever.** One bad pointer takes down the system.
- Relocation and symbol binding are intricate and a rich source of subtle bugs.

4.2 Option B — bytecode virtual machine (selected)

Applications are compiled to a bytecode the kernel interprets. Every memory access executes as a VM instruction, so the interpreter can bounds-check it.

- **Isolation becomes achievable in software** — the VM refuses out-of-range access rather than performing it. This recovers, in the interpreter, the guarantee the silicon will not give.
- The instruction set is ours, so it can be kept small and auditable.
- **Preemption becomes trivial.** The dispatch loop can check a quantum counter between instructions; an application can always be interrupted at a known-safe boundary, with no native context switch required for application tasks.
- Slower than native — the interpretation overhead is real and unmeasured.
- Costs flash for the interpreter, and requires a compiler or assembler for the application format.

Decision: Option B. The isolation argument is decisive. An operating system whose applications can silently corrupt each other is firmware with extra steps, and on hardware without an MMU the VM is the only mechanism that can deliver the property.

The preemption consequence is worth stating separately because it simplifies the kernel: native preemptive context switching is needed only for kernel and driver tasks, not for applications.

5. Component inventory

Component	Layer	Status
Entry stub (`start.S`)	L2	Complete, verified
UART console (`uart.c`)	L3	Minimal; output only, no configuration
Kernel entry (`kmain.c`)	L2	Placeholder with self-checks
Link map (`linker.ld`)	L2	Complete for RAM-only images
Build pipeline (`build.ps1`)	—	Complete
Timer / interrupt controller	L2	**Not started (M1)**
Scheduler and context switch	L2	**Not started (M2)**
Heap allocator	L2	**Not started (M3)**
Bytecode VM	L4	**Not started (M4)**
Display, touch, SD drivers	L3	Not started

6. Explicit non-goals

- **POSIX compatibility.** Nothing here targets a standard API.
- **Binary compatibility with ESP-IDF or Arduino.** Existing sketches will not run.
- **Reuse of the BibleOS/The Word Device codebase.** That project is LVGL-on-Arduino firmware; it is a separate artifact and is not a migration target.
- **Memory protection between native tasks.** Impossible on this silicon; protection applies to VM applications only.

7. Open architectural questions

1. **SMP.** The second core is currently unused. Whether the scheduler becomes SMP-aware or core 1 is dedicated to drivers is undecided and should be settled before M2 fixes the scheduler's shape.
2. **Flash execute-in-place.** M0 runs entirely from RAM. The kernel will eventually exceed 128 KB of IRAM and require cache configuration — a dependency on L1 behaviour that has not been examined.
3. **Application toolchain.** The VM needs a producer: an assembler for the bytecode, a compiler from a higher-level language, or both. Unscoped.
4. **Persistence.** SD is present but no filesystem layer is designed. FAT via an existing implementation would contradict "from scratch"; writing one is a substantial subproject.

8. References

- UM-NATOS-002 — Boot Chain and Image Format
- UM-NATOS-003 — Xtensa ABI Selection
- UM-NATOS-004 — Memory Map and Allocation Policy

- ESP32 Technical Reference Manual, "System Address Mapping"